

Group #: 18

CSE 110 In-Class Activity
Exploring the Not So "Hidden" SWE Rubric

Topic	Bad	Good
Repo Usage (over time) tim	• Inconsistent commits • dumps/clumps of contributions • long period of no activity	• consistent commit activity • history reflects actual development • even contribution between team
Repo Consistency (naming, style) tim	• no branch convention ("fix", "test1", "timsbranch") • lazy, inconsistent commit message • no filename convention	• consistent naming convention/tags, ("feature/", "fix/", "chore/") • style is enforced via linter or convention doc
Commits (frequency & messages) tim	• One big commit lots of changes and possible conflicts • uninformative messages ("final", "done")	• Frequent commits for each bit of progress done • short, descriptive commit messages that explain changes
Pull Request (size, frequency) olivia	PRs merged without comments or approval PRs wait for a long time before they are reviewed Only one person does all the review	Every PR requires at least 1 approval to merge PRs reviewed with one day and shared among team members Small and focused PR (under 300 LoC)
Pull Request Reviewing	• vague comments that do not explain the changes • nitpicking unimportant issues • reviewing too late or using harsh tone	• clear and respectful feedback • explains why a change is suggested • actionable and concise comments • timely feedback
Branching Use Anvay	• everything committed directly to main • ambiguous branch names • one long-lived branch that diverges so far from main it becomes a nightmare to merge	• each feature or fix gets its own branch • branch names match the work and are unambiguous • branches are short-lived
CI/CD Pipeline Anvay	• no pipeline exists • pipeline exists but nobody fixes it when it breaks • tests only run manually • broken builds get merged anyway	• pipeline runs automatically on every PR • build must pass before merge is allowed • pipeline catches failures before they hit main • status visible to whole team
Issue Tracking Anvay	• no issues created • issues are vague • work happens with no corresponding issue • issues never closed	• every piece of work has a linked issue • issues have clear titles and descriptions • issues are closed when work is merged • labels used to categorize
Repo Org (entry point)	no obvious entry file — unclear how to run the project entry point buried deep inside folders multiple possible entry points -- confusing entry point crashes immediately due to missing environment or dependencies	root directory contains a clear entry file (e.g., main.py, index.js, app.py) README explicitly states where to start entry point is minimal and only boots the application running the entry point works without editing any paths or configs
Repo Org (clarity)	folders named vaguely content repeat files with unrelated purposes live in the same folder inconsistent naming (camelCase here, snake_case there, no pattern)	folder names intuitive (e.g., src/, tests/, docs/, scripts/) no duplicate or unused folders (e.g., old/, backup/, archive/) each file has a single, obvious purpose based on its location naming conventions are consistent across repo

Members Present: Olivia, Aila, Aaron, Brendan, Tim, Daniel, Anvay

Topic	Bad	Good
Backlog Use	• outdated or unclear tickets • poor prioritization • large backlog with duplicate tickets	• clear, prioritized backlog items • requirements and criteria are defined • regular maintenance of the backlog
Sprint Planning and Estimation	• overcommitting work in sprints • estimates made without clear specifications • goals not being consistent	• realistic estimations based on constraints • clear goals that are prioritized and communicated • team participates in discussion and decision-making
Sprint Execution & Tracking	• poor tracking which leads to last-minute emergency fixes • tasks not having deadlines and being in progress for too long	• consistent progress through some commonly used tool such as github actions/issues • regular communication during meetings • blockers are resolved and discussed frequently
Sprint Review / Retrospective olivia	retrospective is skipped no actionable items come out people blame each other	everyone shares at least one thought actionable items come out focus on process, not people psychological safety time boxed and different formats are used
Daily Stand-Ups olivia	takes too long (over 15 minutes) issues are raised but not followed up people are multi-tasking, not listening goes into technical dives	takes 10-15min did, do, issues format blockers are addressed immediately after people are engaged meeting supplements further communication
Code Consistency	• mixed coding styles across the files • inconsistent naming of variables • having different AI do different styles	• consistent naming conventions and formatting • shared standards followed by all members • predictable structure that is readable
Code Commenting	• redundant comments that explain self-explanatory code • outdated comments that are confusing	• comments explain the intent and logic of the code • documentation style is updated with code • clear descriptions that make it easy to onboard
Code Decomposition	• large chunks of code that are hard to understand • over-engineering with unnecessary abstraction	• small, reusable components • clear separation of different functions that allow for testing and maintenance.
Code Linting and Style Enforcement	• inconsistent linting rules • excessively strict rules that makes it slow to write code • devs ignoring linting warnings	• automated linting tools are used as devs code • style and rules are defined and enforced in CI
Unit Testing olivia	No tests Test size too big, tests too many features Tests pass but are never run Tests have vague names	every new feature has corresponding test units tests are automated to run on each PR tests run independently and checks exactly one behavior
E2E Testing	• flaky tests that don't actually do anything • slow test execution that delays development	• tests validate real user workflows • they are automated and integrated within the CI • stable test environments
Hand Testing (Performed, Documented, ...)	• testing done inconsistently or skipped • no documentation of the testing make them not as efficient • bugs reported without details or comments are too general	• manual testing is done before release • clear test conventions are documented and followed • tests include edge cases and are rigorous enough to be effective

Members Present: Olivia, Aila, Aaron, Brendan, Tim, Daniel, Anvay

Topic	Bad	Good
"Dog Fooding"	- internal users may not represent real customers - bias towards known workflows prevent bugs to be seen	- helps team experience the product - connects team with the product - fast feedback on usability and functionality
Stakeholder Feedback daniel	Too much stakeholder feedback can distract the team, and may be hard to prioritize which feedback the team should work on. The feedback could also be too much work for the team to do within the scope and timeframe of the project.	Stake holder feedback can bridge the gap between technical implementation and real-world utility, ensuring the product solves actual problems rather than just functioning according to initial requirements
Outside User Acceptance Testing	- feedback is unclear and not documented - Users ask for unrealistic features that are not doable with given constraints	- Feedback comes from real user workflows - finds issues developers might be too biased to notice - Provides good feedback and allows for user-centered goal of the project.
Team Communications (Ad hoc) daniel	Random communication can become disorganized. Important decisions might get lost in chats, and some teammates may miss updates.	Quick informal communication helps solve small problems fast. It lets teammates ask questions, clarify tasks, and unblock each other without waiting for a meeting.
Team Communications (Meetings)	with larger groups, it is often difficult to align everyone's times to find time to meet. Every meeting might not be needed and thus some meetings might feel less efficient	It allows everyone to regularly be on the same page and have a dedicated time where people can share their progress, blockers, etc.
Other Team Dynamics daniel	Team members may have different working styles, skill levels, or expectations. This can cause conflict, uneven participation, or confusion about responsibilities.	A good team dynamic can make a project more enjoyable and can get more work done such as compromising with the team, respecting others ideas, etc.
Workload Distribution	- uneven task allocation - same people always doing the same work which leads to low bus-factor - lack of clear distribution	- tasks are balanced across members - the responsibilities align with people's skills and strengths, or areas where they want to grow in - clear accountability
Workload Pace	- constant urgent situations and doing things last minutes - unrealistic timeline with expectations and set of constraints - rushing development and not leaving time to test	- realistic and communicated deadlines - delivering consistently and in small increments that allow for thorough testing before deadline - routine sprint and team meetings to get everyone on the same page
Other Things You Think Of TEAM DYNAMIC	- team members do not know each other - team members do not respect each other's times and spaces - team is too casual with each other and therefore there is no sense of urgency or responsibility to the team/product	- there is a good balance of friendship and professionalism - there are no cliques within the group that leave people out - the tone is always respectful and professional - people feel comfortable to ask questions and communicate their emotions

Is this an analog range (Absent, Striving, ... Perfect) or more binary? analogWill an SWE "rubric" be consistent across teams/organizations/industries? Y N

Explain why or why not?

Because a lot of the standards and expectation should be set within the team itself and discussed as a group. This way, people will feel more inclined to follow the rubric and write them in a way that matches the dynamic, availability, constraints, and skillsets of the group

Members Present: Olivia, Aila, Aaron, Brendan, Tim, Daniel, Anvay

Group #: 18

Product Quality Characteristic	Importance (1 - 5)	Bad	Good
Functional Suitability	5	This requires a lot of testing and will be time consuming. This also requires user feedback which means it will take even longer and can be subjective and requires a hard rubric to make sure that we understand what qualifies as functional	But making sure that we are taking functional suitability as a top priority makes sure that we continue to keep our product on check and are not deviating from our original idea of our project
Performance Efficiency	4	Solely focusing on performance efficiency can lead to a rabbit hole where we focus solely on one features efficiency and we end up not making the rest of the efficiency. Or that working on all performances it becomes too time consuming	Although we have to make sure our performance efficiency is high because a low performance efficiency means users are less likely to use it because it takes too long.
Compatibility (interop)	3	If we work too much on making sure that it works on all devices it might take away from making sure that the core functions are working. Making sure it fully works on one spot is better so than it will be easier to move core functions on another place.	While worrying too much on compatibility may be time consuming it is still needed to take in a fair amount of customers and by making it work across devices will bring in more users.
Usability	5	Usability is a main thing of how the app can function but getting the bare minimum usability is needed at first before working on more because working too much on may not allow for backend functions to be tested till later.	But making sure that usability is a top priority is needed as it is the main way that users can access our features. If the usability is bad but the backend is good it won't last because user value the usability over the amount of features we may have.
Reliability	5	Making sure that the feature is reliable is something is time consuming. It requires a lot of testing to make sure that app will always work.	Having reliability be the fore front of our project is very much needed as our app needs to constantly be able to detect the text to make it editable and if that feature doesn't work than our whole app falls apart. So making sure features like that always work is needed and users are more likely to recommend it
Security	3	Security's importance depends on the app for us it's not that important as we don't hold too much information so focusing a lot on security would be overkill for this kind of app.	But we do need to focus on security for memes made as we are dealing with certain people's faces and if we lose those then it would become a issue so security is good to have but too much will be overkill.
Maintainability	4	For our app we want it to be long lasting app but it also is a meme generator app that isn't a very necessary so making it maintainable isn't something that needs to be a top importance.	But when working on the app trying to think of maintainability will allow us to make it easier to add in new features in the future because making sure our core features are always working well means we can add new features. It also makes sure our app lasts for long time which if we want to go from a prototype to an actual app we need it to be maintainable.
Portability	3	For our app working too much on the portability of the app will take too much time away from creating the core functions in the little time we have. Yes do we want the feature on phones yes but for now we can just make it as a website that looks nice on phones as trying to implement it with ios or Instagram may take too much time and effort	Getting the app the work on phones and making it a more portable type of feature would be great for our users as we are a meme generator app. Most people will want to be using this app on their phones so being portable in that sense is needed for our app to survive in a society where everything is on phones.

Is it possible to put a 5 on all these characteristics reasonably? Y N

What Prof favorite word helps us accept this? tradeoff

Guess the class split % between process / product : 35 / 65
 Guess industry split % between process / product : 60 / 40

Will the split be consistent in the industry across organizations? Y N Explain

What -ility might overwhelm all in industry? Maintainibilty Viability

Members Present: Olivia, Aila, Aaron, Brendan, Tim, Daniel, Anvay

If a software project fails despite having good process, what could have happened?

The process could have taken a long time, to the point where the need for that project changed, new projects beat them to it.

If a software project succeeds despite having poor process, what could have happened?

the team might have gotten lucky in the sense that the domain of the project desperately needed a software and the project came to be at the right place at the right time.

If a software product succeeds despite having poor quality characteristics, what could have happened?

It may have had a strong "X Factor", meaning that something outside the normal software quality made people still want it. For instance, it may have solved a very important problem, had great timing, strong marketing, network effects, loyal users, low cost, or lack of competitors. - brendan

If a software product fails despite having good quality characteristics, what could have happened?

It may have LACKED that "X Factor". Even if the software was reliable, usable, maintainable, and well-built, it may have not had the special ingredient that made users actually want it. It might not have matched what they wanted, entered the market too late, had poor marketing, had too much comp, or failed to create that demand through excitement. - brendan

What does the idea of "X Factor" mean to you, given those last four questions?

To me, the "X Factor" is the hard to measure, extra sauce that affects success beyond technical quality. It can include timing, usefulness, user demand, emotional appeal, branding, market fit, luck, or how well the product connects with people. It is so crucial to success and is important to taking your product to the next level. - brendan

Should evaluations in academia acknowledge "X Factor"?

Y

N

Explain why you answered that way with a specific scenario, if possible.

ABSOLUTELY, because real world success is not only about technical quality. A product is not successful because of its quality alone. It needs to be both well-made and have that X-Factor. A quality product will fail if people don't need or want it and the flip side is true. Thus, due to its importance, it should definitely be acknowledged in academia. - brendan

Members Present: Olivia, Aila, Aaron, Brendan, Tim, Daniel, Anvay